

Základy návrhu algoritmů

Základy programování

Matouš Cejnek

U12110, FS, ČVUT v Praze

Co si dnes odnesete?

- Jak podmínky a cykly řídí **tok programu**?
- Jak z programu odhadnout **počet operací**?
- Co říká časová složitost a značení **Big O**?
- Jak číst a kreslit **flowchart**?
- Jak souvisí flowchart s **UML activity diagramem**?

Na co navazujeme a proč to potřebujeme?

- Algoritmické myšlení lze rozvíjet bez ohledu na znalost konkrétního programovacího jazyka.
- Začneme plánováním řešení **dříve**, než přejdeme k zápisu kódu.
- Stejný problém může mít více správných, ale různě efektivních řešení.
- Diagram pomáhá o programu komunikovat bez závislosti na syntaxi jazyka.

Algoritmus není totéž co program

Algoritmus

Konečný a jednoznačný postup, který převádí vstup na požadovaný výstup.

- **Algoritmus** popisuje myšlenku řešení.
- **Program** je realizace algoritmu v konkrétním jazyce.
- Jeden algoritmus lze zapsat slovně, pseudokódem, diagramem i kódem.

Co musí být v zadání problému?

Vstup

- Jaká data dostáváme?
- Jak jsou velká?
- Jaká omezení platí?

Výstup

- Co přesně vracíme?
- Jak poznáme správnost?
- Co se stane na hranicích?

Nejdříve specifikace, potom řešení

Bez přesného vstupu a výstupu nelze spolehlivě posoudit správnost algoritmu.

Tři stavební kameny toku programu



- **Posloupnost:** příkazy se vykonají v pořadí.
- **Větvení:** další krok závisí na podmínce.
- **Cyklus:** část postupu se opakuje.

Podmínka if: vyber jednu cestu

```
if temperature < 0:  
    print("Mrzne")  
else:  
    print("Nemrzne")
```

- Podmínka má hodnotu True nebo False.
- Vykoná se právě odpovídající větev.
- Po větvení program pokračuje společným dalším krokem.

Více možností: if – elif – else

```
if score >= 90:  
    grade = "A"  
elif score >= 75:  
    grade = "B"  
else:  
    grade = "C"
```

Pořadí podmínek je součástí algoritmu

Python testuje podmínky shora dolů a použije první pravdivou větev.

Cyklus `while`: opakuj, dokud podmínka platí

```
n = 8
while n > 1:
    n = n // 2
    print(n)
```

- Podmínka se kontroluje **před každým průchodem**.
- Tělo se nemusí vykonat ani jednou.
- Některý krok musí směřovat k ukončení cyklu.

Kdy se `while` nezastaví?

- Řídicí hodnota se vůbec nemění.
- Mění se opačným směrem, než potřebuje podmínka.
- Některá větev přeskočí aktualizaci hodnoty.
- Ukončení závisí na vstupu, který nemusí nikdy nastat.

Invariant a varianta cyklu

Invariant zůstává pravdivý; **varianta** se mění směrem k ukončení.

Cyklus for: projdi známou kolekci

```
numbers = [3, 8, 1, 6]
total = 0

for number in numbers:
    total += number
```

- Proměnná `number` postupně získá každý prvek.
- Počet průchodů je dán počtem prvků kolekce.
- V Pythonu `for` znamená „pro každý prvek“.

for, nebo while?

Použijte for

- pro každý prvek,
- známý počet opakování,
- průchod přes range.

Použijte while

- opakování do splnění podmínky,
- počet kroků předem neznáme,
- stav řídíme sami.

break a continue mění běžný tok

```
for number in numbers:
    if number < 0:
        continue          # dalsi prvek
    if number == 0:
        break             # konec cyklu
    total += number
```

- continue přeskočí zbytek aktuálního průchodu.
- break ukončí nejbližší cyklus.
- V diagramu musí být obě změny toku viditelné šipkou.

Proč počítat operace?

- Čas na stopkách závisí na počítači, jazyce i aktuálním zatížení.
- Počet kroků algoritmu lze porovnávat nezávisle na konkrétním stroji.
- Zajímá nás, jak práce roste s velikostí vstupu.

Velikost vstupu n

Například počet prvků seznamu, počet znaků textu nebo počet vrcholů grafu.

Co budeme považovat za jednu operaci?

- přiřazení nebo aritmetická operace,
- porovnání a vyhodnocení jednoduché podmínky,
- přístup k jednomu prvku seznamu,
- jednoduchý výpis — v našem zjednodušeném modelu.

Je to model

Skutečné operace netrvají stejně dlouho. Model musí být jednoduchý, ale použitý konzistentně.

Příklad: přesný počet kroků

```
total = 0                # 1x
for number in numbers:   # n prvků
    total = total + number # n-krát
print(total)             # 1x
```

- Zjednodušeně: $T(n) = 1 + n + n + 1 = 2n + 2$.
- Přesný výsledek závisí na tom, co zahrneme do jedné operace.
- Důležitější než koeficient je zde **lineární růst**.

Podmínka: kterou větev počítáme?

- **Nejlepší případ:** vstup vyvolá nejméně práce.
- **Nejhorší případ:** vstup vyvolá nejvíce práce.
- **Průměrný případ:** potřebujeme předpoklad o rozdělení vstupů.

Běžná dohoda

Není-li uvedeno jinak, často analyzujeme nejhorší případ: dává bezpečnou horní mez.

Předčasný konec: nejlepší a nejhorší případ

```
for number in numbers:
    if number == target:
        return True
return False
```

- Cíl je první prvek: přibližně 1 kontrola.
- Cíl je poslední nebo chybí: přibližně n kontrol.
- Nejlepší případ $\Theta(1)$, nejhorší $\Theta(n)$.

Dva cykly za sebou se sčítají

```
for x in numbers:           # n
    print(x)

for x in numbers:           # n
    print(x * x)
```

$$T(n) \approx n + n = 2n \quad \Rightarrow \quad \Theta(n)$$

Dvojnásobek práce je stále lineární růst.

Vnořené cykly se násobí

```
for x in numbers:           # n
    for y in numbers:       # n pro každé x
        print(x, y)
```

$$T(n) \approx n \cdot n = n^2 \Rightarrow \Theta(n^2)$$

Každá z n vnějších iterací spustí celý vnitřní cyklus.

Půlení problému vede k logaritmu

```
while n > 1:  
    n = n // 2
```

Po k krocích zbývá přibližně $n/2^k$. Končíme, když

$$\frac{n}{2^k} \leq 1 \quad \Rightarrow \quad k \geq \log_2 n.$$

Proto je počet průchodů $\Theta(\log n)$.

Od přesného počtu k řádu růstu

- $3n + 7$, $100n + 2$ i n rostou pro velké n lineárně.
- Konstantní násobky a nižší členy nemění dominantní růst.
- Asymptotická analýza se ptá: **co převládne pro velké vstupy?**

$$4n^2 + 3n + 12 \rightsquigarrow n^2$$

Jak číst Big O?

- Big O popisuje, **jak roste množství práce**, když zvětšujeme vstup.
- Neříká přesný počet sekund ani rychlost konkrétního počítače.
- Zajímá nás hlavní trend, proto zanedbáváme konstanty a méně významné části.

Příklad

$3n + 7$ kroků zapisujeme jako $O(n)$: s počtem prvků roste práce přibližně lineárně.

Tři jednoduché příklady

Postup	Práce pro n prvků	Zápis
přístup na známý index	jeden krok	$O(1)$
jeden průchod seznamem	nejvýše n kroků	$O(n)$
porovnání všech dvojic	přibližně n^2 kroků	$O(n^2)$

Otázka při odhadu

Kolikrát se provede nejčastěji opakovaná operace, když má vstup n prvků?

Typické třídy růstu

Složitost	Název	Typický příklad
$O(1)$	konstantní	přístup na známý index
$O(\log n)$	logaritmická	opakované půlení
$O(n)$	lineární	jeden průchod seznamem
$O(n \log n)$	lineární	efektivní třídění
$O(n^2)$	kvadratická	všechny dvojice prvků
$O(2^n)$	exponenciální	výběr všech podmnožin

Malý vstup může klamat

- Algoritmus s $1000n$ může být pro malé n pomalejší než algoritmus s n^2 .
- Pro dost velké n však kvadratický růst lineární překoná.
- Big O neříká skutečný čas ani konstanty.
- Po asymptotické analýze má smysl program také měřit.

Co je flowchart?

Vývojový diagram (flowchart)

Grafické znázornění kroků algoritmu a toku řízení mezi nimi.

- Uzly představují kroky, rozhodnutí nebo vstup a výstup.
- Orientované šipky určují pořadí vykonávání.
- Diagram zachycuje logiku; nemusí kopírovat každý řádek kódu.

Mermaid je zápis, flowchart je diagram

- **Flowchart** je způsob modelování algoritmu.
- **Mermaid** je textový jazyk, který diagram vykreslí.
- Tentýž flowchart lze nakreslit rukou, v editoru nebo zapsat v Mermaid.

Příklad zápisu

```
flowchart TD      start([Start]) -> process[Zpracuj data]
```

Značka: začátek a konec



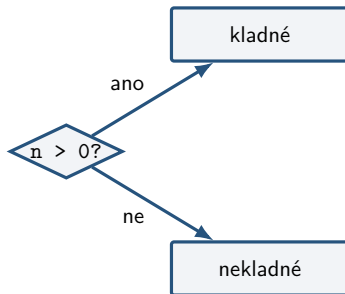
- Zaoblený terminátor označuje vstupní nebo koncový bod toku.
- Diagram má mít jasný začátek a alespoň jeden konec.
- Mermaid: `start([Start]), stop([Konec])`.

Značka: proces neboli akce

```
total = total + number
```

- Obdélník představuje výpočet, přiřazení nebo jiný krok.
- Text má být krátký, konkrétní a ve slovesném tvaru nebo jako příkaz.
- Mermaid: `step[Sečti hodnoty]`.

Značka: rozhodnutí



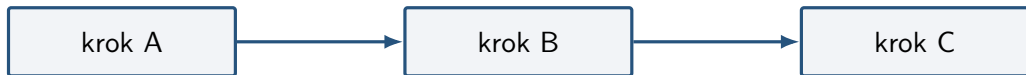
- Kosočtverec obsahuje otázku nebo logickou podmínku.
- Každá odchozí hrana musí mít jednoznačný popis.
- Mermaid: `decision{n > 0?}`.

Značka: vstup a výstup



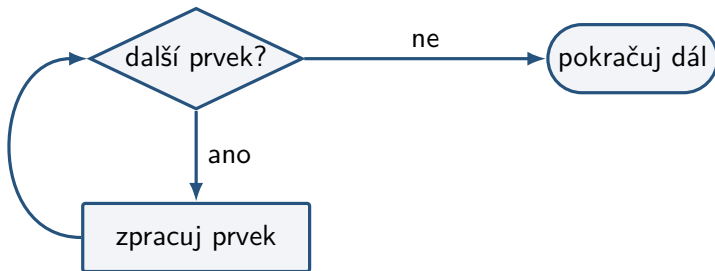
- Rovnoběžník tradičně označuje načtení nebo výpis dat.
- Odlišuje komunikaci s okolím od vnitřního výpočtu.
- Mermaid podporuje tvar rovnoběžníku; běžný obdélník je také čitelný, pokud jsme konzistentní.

Značka: šipka a spojení toku



- Hrot šipky určuje směr řízení.
- Tok obvykle vedeme shora dolů nebo zleva doprava.
- Křížení hran omezujeme; složitý obraz často signalizuje příliš mnoho detailů.

Jak diagram vyjádří cyklus?



- Rozhodnutí testuje pokračování cyklu.
- Zpětná šipka vrací tok k další kontrole podmínky.

Další tvary: užitečné, ne vždy nutné



- Válec: uložená data nebo databáze.
- Obdélník s dvojitým okrajem: předem definovaný proces / podprogram.
- Pro jednoduché algoritmy obvykle stačí terminátor, proces, rozhodnutí, I/O a šipky.

Flowchart versus UML activity diagram

Flowchart

- obecná a pružná notace,
- tradiční tvary pro proces a I/O,
- vhodný pro vysvětlení algoritmu.

UML activity diagram

- standardizovaná UML notace,
- akce, rozhodnutí a řídicí toky,
- umí paralelní větve a odpovědnosti.

Pro naše úlohy

Význam je velmi podobný. Důležitější než název je čitelný a konzistentní tok.

Jak se liší základní značky UML aktivit?



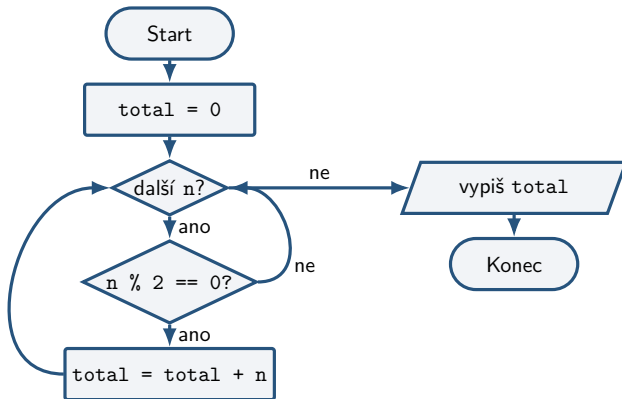
- Plný kruh: počáteční uzel; terč: konec aktivity.
- Zaoblený obdélník: akce; kosočtverec: rozhodnutí nebo sloučení.
- Podmínky na hranách se často zapisují jako [ano] a [ne].

Od programu k diagramu: nejdříve bloky

```
total = 0
for n in numbers:
    if n % 2 == 0:
        total += n
print(total)
```

- 1 inicializace,
- 2 kontrola dalšího prvku,
- 3 rozhodnutí sudé / liché,
- 4 případná aktualizace,
- 5 výpis po skončení cyklu.

Od programu k diagramu: výsledný tok



Časté chyby v diagramech

- Rozhodnutí nemá popsané odchozí větve.
- Cyklus nemá zpětnou hranu nebo cestu ven.
- Výpis uvnitř cyklu je omylem zakreslen až za cyklem.
- `break` vede k další iteraci místo za cyklus.
- Diagram mechanicky kopíruje každý řádek a přestává být čitelný.

Jak kontrolovat správnost diagramu?

- 1 Zvolte malý konkrétní vstup.
- 2 Sledujte šipky a zapisujte hodnoty proměnných.
- 3 Stejný vstup projděte řádek po řádku v programu.
- 4 Porovnejte pořadí kroků, větve a výsledný výstup.
- 5 Zopakujte pro hraniční vstup: prázdný seznam, nulu, první shodu.

Jedna úloha, tři pohledy

- **Kód** přesně říká, co vykoná počítač.
- **Diagram** ukazuje strukturu toku a usnadňuje komunikaci.
- **Analýza složitosti** říká, jak množství práce roste se vstupem.

Dobrý návrh propojuje všechny tři

Dokážeme vysvětlit správnost, zakreslit tok a odhadnout počet operací.

Co si zapamatovat?

- `if` větví tok; `for` prochází prvky; `while` opakuje podle podmínky.
- Sekvenční cykly sčítáme, vnořené cykly typicky násobíme.
- Big O je asymptotická horní mez; Θ vyjadřuje těsný řád růstu.
- Flowchart tvoří uzly a orientované šipky; značky používejte konzistentně.
- UML activity diagram je formálnější příbuzná notace, nikoli jiný algoritmus.

Otázky?